Local File Search - Manual Test Plan

Generated 21 August 2026 | Fill in **Result** and **Remark** as you go. Rows marked ***** are the highest-value checks if you are short on time.

Thresholds the app uses - judge a result against these, not against a gut feeling.

0.55 relevance floor: below this a result is not shown at all. **0.62** confidence: below this the result still appears but is labelled *weak match*. **0.95** of the top hit: anything scoring less than 95% of the best result is trimmed. **10** results per page.

When reporting a problem, note: **the query**, **what came top**, **its similarity number**, and **what you expected instead**. The number is what separates a ranking problem from a threshold problem.

A. Basic sanity

Does the plumbing work at all.

#	What to do	Expected	Result	Remark
A1	Copy an exact sentence out of one of your PDFs and search it.	That file at rank 1, **strong match (0.72 or above)**, no weak-match caveat.		
A2	Search a document's actual title.	That file at rank 1.		
#	**What to do**	**Expected**	**Result**	**Remark**
A3	Search gibberish: `asdkjhaskdjh qwe`.	**0 results** and the message "Nothing in your library matches that closely".		
A4	Search only punctuation, e.g. ???, or a single space.	Nothing returned, no error, no crash.		
A5	Search a very common word: *the*, *and*.	0-2 weak results. Many results here means the floor is too low for your library.		
A6	Click **Open** on any result.	The original file opens.		
A7	Read the grey line under a result heading.	Reads like *PDF - document text - page 9 - strong match (0.78)*. No emoji. Page number present for PDFs.		

B. Semantic retrieval - the actual point

The system should win where keyword search loses.

#	What to do	Expected	Result	Remark
B1*	**Describe a document's content using no words that appear in it. E.g. for DevOps notes: *how do teams ship code automatically*.**	Correct file at rank 1. This is the whole product.		
B2	Misspell a key term: *recipie*, *kubernets*, *machene learning*.	Still finds it. Score drops roughly 0.10.		
#	**What to do**	**Expected**	**Result**	**Remark**
B3	Ask the same thing as a question, then as a bare phrase.	Both find the same file; scores may differ by 0.02-0.05.		
B4	Search a concept **implied but never stated**, e.g. *is this candidate a beginner or experienced?* against a resume.	Should find the resume. Genuinely hard - a miss is a limitation, not a bug.		
B5	Search something you know is in **exactly one** file.	1 result. More than one means the cutoff is too loose.		
B6	Search a plausible topic that is **not** in your library, e.g. *tax filing deadlines*.	0 results, or 1 result flagged as weak.		

C. Multiple files on the same subject

The most informative group. Set up 3-4 files on one subject first (several units of one course, several notes on one topic).

#	What to do	Expected	Result	Remark
C1*	**Search the shared topic, e.g. *devops notes*.**	**All the related files appear**, one entry each, with similarities within about 0.05 of each other. The 0.95 relative cutoff keeps genuinely close files.		
C2	Look at the ordering in C1.	The file whose **single best passage** matches most closely wins - **not** the one with the most matches. Long documents must not win on length alone.		
#	**What to do**	**Expected**	**Result**	**Remark**
C3	Search something present in **only one** of those files.	That one alone, or clearly first. Separates real discrimination from "always returns the whole topic cluster".		

#	What to do	Expected	Result	Remark
C4	Expand "*N more matches in this file*" on a result.	Up to 5 supporting passages, best first, each with its own page number.		
C5	Put the **same file twice under different names** in a folder, then index it.	Only **one** entry. The second is reported as already present. Identity is the content hash, not the filename.		
C6	Put **v1 and v2** of the same document in (a few edits apart).	**Both** appear, with near-identical scores. Different bytes means different files.		
C7	Search a phrase that appears in the **header or footer of every page** of one long PDF.	That file must appear **once**, not twenty times.		
C8	Untick *One result per file* and repeat C1.	Many more results, at passage level, numbered continuously.		

D. Scans and OCR

Image understanding is currently off; OCR is not affected and still runs.

#	What to do	Expected	Result	Remark
D1	Index a **scanned PDF** (photographed pages, no text layer). Search its printed content.	Found. The match is labelled **"text read from an image"**.		
D2	Search a **number** from a scan: an invoice total, a date, an ID.	Found - OCR preserves digits.		
#	**What to do**	**Expected**	**Result**	**Remark**
D3	Index a **photo of a document** or a screenshot. Search its text.	Found via OCR.		
D4	Index a photo with **no text at all** (a landscape, a person). Search by appearance.	**Not findable.** Expected - image understanding is off until the GPU machine.		
D5	Compare a scan against a text PDF of similar content.	The scan usually scores lower; OCR introduces noise.		
D6	Try a rotated or skewed scan.	Tesseract will struggle. Worth noting - PaddleOCR on the GPU box handles this better.		

E. Filters and pagination

#	What to do	Expected	Result	Remark
E1	Run a broad query, then switch *Everything* to *Documents*.	Same or fewer results, PDFs and DOCX only.		
E2	Switch to *Images*.	Image files only. Likely **0** unless you have indexed photos containing text.		
#	**What to do**	**Expected**	**Result**	**Remark**
E3	Get a query returning more than 10 results (untick *One result per file* to force it).	Footer reads *Showing 1-10 of N - page 1 of X*. Previous is disabled.		
E4	Click **Next**.	Results numbered **11, 12...** with no repeats from page 1.		
E5	Go to the last page.	Next is disabled. A partial page is fine.		
E6	Go to page 3, then **change the filter**.	Jumps back to **page 1** - not an empty screen.		
E7	Go to page 3, then **edit the query**.	Resets to page 1.		
E8	Change results-per-page from 10 to 5.	Page count doubles and you return to page 1.		

F. Library and folder indexing

F3 is the single most important check on this page.

#	What to do	Expected	Result	Remark
F1	Paste a folder path into the Library tab.	A preview appears: "N supported file(s) found", **before** you commit.		
F2	Press **Index folder**.	Progress bar advances, then "Done" with a file and passage count.		
#	**What to do**	**Expected**	**Result**	**Remark**
F3*	**Check the source folder afterwards in Explorer.**	Every file still there, same size, same timestamp. **Indexing must never modify, move or delete your originals.**		
F4	Index the **same folder again**.	"0 added, N already present." Nothing duplicated, nothing deleted.		

#	What to do	Expected	Result	Remark
F5	Index with **Include subfolders** off, then on.	Off indexes the top level only; on picks up nested files.		
F6	Index a folder with junk mixed in (.zip, .exe, .txt).	Silently skipped and counted as unsupported.		
F7	Enter a path that does not exist.	Clear message: "No such folder: ...".		
F8	Enter a **file** path instead of a folder.	"That is a file, not a folder."		
F9	Point at an empty folder.	"No supported files" and the Index button stays disabled.		
F10	Paste a path copied with Windows *Copy as path* (includes quotes).	Works - the quotes are stripped.		
F11	Search **while indexing is still running**.	Search still responds. Results grow as processing finishes.		
F12	**Remove** a file in the Library tab, then re-run a query that found it.	Gone from the results immediately.		
F13	Index a folder, then **restart the backend** mid-processing.	It resumes on its own. No re-import needed.		

G. Edge cases

#	What to do	Expected	Result	Remark
G1*	**Index a folder containing a corrupt or truncated PDF.**	That file shows *failed* with a reason. **Every other file still indexes.**		
G2	Index a **password-protected PDF**.	Fails with "This PDF is password protected". Others unaffected.		
#	**What to do**	**Expected**	**Result**	**Remark**
G3	Index a file larger than **200 MB**.	Counted as "over the size limit", not ingested.		
G4	Two different files with the **same filename** in different folders.	Both indexed - identity is content, not name.		
G5	A document that is **one line long**.	Indexed, provided it has at least two distinct words.		
G6	A document that is **almost entirely numbers or tables**.	May be filtered as low-information and be **unsearchable**. Expected - but flag it if it hits something you care about.		

#	What to do	Expected	Result	Remark
G7	Paste a **whole paragraph** as the query.	Works, and often scores better than a short query.		
G8	Search a **non-English** document in its own language.	Poor results. The text model is English-only - known limitation.		
G9	Filenames with spaces, #, &, or accented characters.	Index and open correctly.		
G10	A PDF whose metadata title is junk, e.g. "(anonymous)" or a patent number.	Result is headed with the **filename**, underscores shown as spaces.		
G11	Search an exact filename, e.g. *Unit 3 notes UI_UX.pdf*.	Should find it, but matching is by content so it may score weak.		
G12	Index a folder, then **move or rename the original** outside the app.	Still searchable and still opens - the app holds its own copy.		

H. Known limitations - confirm, do not report as bugs

These are measured properties of the model, not defects. Worth seeing once so you know their shape.

Behaviour	What you will see	Seen?	Remark
Short or acronym queries	e.g. *EEIM notes*, *UI UX*: the correct file, but scoring around 0.60, so it carries the *weak match* caveat even when right.		
Short queries score high against everything	*quantum chromodynamics*, with nothing relevant indexed, still reaches 0.572.		
No clean threshold exists	Measured over 20 queries: noise ceiling **0.572**, signal floor **0.581** - a gap of 0.009. Some noise gets through; some weak-but-correct hits get flagged.		
Images are inert	Anything needing "what does this picture look like" fails until image understanding is switched on with a GPU.		
First search of a session is slow	About 20 seconds while the text encoder loads. Instant afterwards.		

Short on time? The four highest-value checks are **C1** (several files on one subject), **B1** (semantic match with no shared words), **F3** (originals untouched after indexing), and **G1** (one bad file does not break the batch).